Garantia da Qualidade de Software

Gestão e Qualidade de Software

Professor Daniel Henrique Matos de Paiva

Lista de Exercícios

Esta lista de exercício deve:

- Ser realizada em equipes de até 06 alunos.

- Ser entregue no **prazo** proposto.

- Todos os alunos da equipe devem entregar os exercícios propostos.

- Entregas individuais.

📋 1. Configuração Inicial do Ambiente

Abra o seu terminal (ou o terminal integrado do Visual Studio Code) e siga os passos abaixo para estruturar a nossa solução, os projetos e o repositório Git.

1.1. Inicialização do Repositório Git

mkdir MiniEcommerce

cd MiniEcommerce

git init

Ou crie direto no GitHub o repositório e clone para a sua máquina).

1.2. Criação da Solução e dos Projetos (.NET)

Vamos criar uma solução limpa contendo o projeto principal de console e o projeto de testes unitários:

Criar o arquivo de solução (.sln)

dotnet new sln -n MiniEcommerce

Criar o projeto de console (Aplicação)

dotnet new console -n MiniEcommerce.Console

Criar o projeto de testes unitários (xUnit)

dotnet new xunit -n MiniEcommerce.Tests

Adicionar os projetos à solução

dotnet sln add MiniEcommerce.Console/MiniEcommerce.Console.csproj

dotnet sln add MiniEcommerce.Tests/MiniEcommerce.Tests.csproj

Referenciar o projeto principal dentro do projeto de testes

dotnet add MiniEcommerce.Tests/MiniEcommerce.Tests.csproj reference MiniEcommerce.Console/MiniEcommerce.Console.csproj

1.3. Primeiro Commit (Estrutura Inicial)

Faça o rastreamento dos arquivos criados e realize o primeiro commit seguindo estritamente o padrão **Conventional Commits**:

git add .

git commit -m "chore: initial project structure with console and xunit apps"

🎯 2. Requisitos Funcionais da Aplicação

A aplicação de console deverá permitir:

1. **Cadastro de Produtos:** Nome, Preço e Quantidade em Estoque (com validação para evitar preços negativos ou estoque menor que zero).
2. **Gerenciamento de Carrinho de Compras:** Adicionar produtos ao carrinho informando a quantidade desejada, validar se há estoque disponível e remover itens se necessário.
3. **Fechamento de Pedido (Checkout):** Processar a compra, realizar a baixa automática no estoque dos produtos adquiridos e calcular o valor total do pedido com base nos itens selecionados.

4. **Catálogo e Consulta:** Exibir a lista de produtos disponíveis com seus respectivos preços e estoques atuais.

🛠️ 3. Roteiro de Desenvolvimento e Padrão de Commits

Para cada etapa concluída, você deve criar testes unitários correspondentes antes de implementar a funcionalidade (TDD básico) e utilizar rigorosamente o padrão **Conventional Commits** (feat, fix, test, refactor, docs, chore).

Etapa 1: Regras de Negócio e Testes para Produtos

- **O que fazer:** Crie a entidade Produto e a classe de gerenciamento de estoque/catálogo. Escreva testes unitários no projeto xUnit garantindo que não seja possível cadastrar produtos com preço negativo ou estoque inválido.
- **Comandos sugeridos:**

dotnet test

git add .

git commit -m "test(produto): add unit tests for product validation rules"

git commit -m "feat(produto): implement product entity and validation logic"

Etapa 2: Carrinho de Compras e Regras de Adição

- **O que fazer:** Implemente a classe CarrinhoDeCompras que permita adicionar itens, calcular o subtotal e verificar limites de estoque. Escreva os testes unitários para cobrir cenários de adição com sucesso e tentativas de adicionar quantidade superior ao estoque disponível.
- **Comandos sugeridos:**

dotnet test

git add .

git commit -m "test(carrinho): add unit tests for cart addition and stock limits"

git commit -m "feat(carrinho): implement shopping cart logic and stock verification"

Etapa 3: Processamento de Pedido e Checkout

- **O que fazer:** Implemente a lógica de fechamento de pedido (Pedido), garantindo que o estoque seja efetivamente decrementado após a confirmação da compra e que o resumo financeiro seja gerado corretamente. Escreva testes unitários cobrindo o fluxo de checkout.
- **Comandos sugeridos:**

dotnet test

git add .

git commit -m "test(pedido): add unit tests for checkout and stock deduction"

git commit -m "feat(pedido): implement order processing and inventory update"

Etapa 4: Interface de Console (Program.cs)

- **O que fazer:** Desenvolva o menu interativo no arquivo Program.cs para que o usuário possa interagir com o e-commerce via linha de comando (cadastrar produtos, listar catálogo, adicionar ao carrinho e finalizar pedido).
- **Comandos sugeridos:**

dotnet run --project MiniEcommerce.Console/MiniEcommerce.Console.csproj

git add .

git commit -m "feat(console): implement interactive menu for e-commerce store"

Etapa 5: Refatoração e Documentação (README.md)

- **O que fazer:** Limpe o código, garanta que todos os testes passem (dotnet test) e crie um arquivo README.md bem estruturado explicando a arquitetura do projeto e como executá-lo.
- **Comandos sugeridos:**

dotnet test

git add .

git commit -m "refactor(core): improve code readability and organize namespaces"

git commit -m "docs: add comprehensive instructions in README.md"

🚀 4. Entrega Final

1. Crie um repositório público no seu GitHub.
2. Conecte o seu repositório local ao remoto e envie o código:

git remote add origin <URL_DO_SEU_REPOSITORIO_GITHUB>

git branch -M main

git push -u origin main

Verifique se o seu histórico de commits está limpo, organizado e totalmente aderente ao padrão **Conventional Commits** executando:

git log –oneline

Bom trabalho a todos!

Qualquer dúvida sobre a estruturação dos testes com xUnit ou sobre os prefixos dos commits, é só chamar.